

Bridge Design Pattern

In this activity, I learned how the Bridge Design Pattern addresses the problem of class explosion when multiple features need to work together. The attendance application needed different report formats, data sources, and compression methods. Without the Bridge Pattern, every combination would require a separate class, such as a JSON report with Gzip compression or a text report with no compression. Adding another format or compression method would increase the number of classes.

To solve this problem, I separated the report formatter from the data source and compression implementations. The `TimesFormatter` class works with the `DataSource` and `Compressor` interfaces, allowing the application to use the API or file data source and either Gzip or no compression. I completed the `setCompressor()` method and created the `TimesheetJsonFormatter` using `json_encode()` with pretty printing. I also demonstrated the different combinations of formats, sources, and compressors, including the difference between compressed and plain output.

Through this activity, I understood that the Bridge Pattern allows two or more parts of a system to change independently. The report format does not need to know how the data is retrieved or how the output is compressed. Instead of creating a separate class for every combination, the system uses composition to connect the components. This makes the application more flexible and reduces duplicated code when new formats, data sources, or compression methods are introduced.